

SEVLEX

CYBERSECURITY, SIMPLIFIED

Vulnerability Assessment Report : http://id.eideasy.com/

Executive Summary

The vulnerability assessment of **http://id.eideasy.com/** identified multiple critical, high, medium, and low-risk security issues impacting application security, user data protection, and overall system resilience.
The system demonstrates **critical weaknesses in CSRF/XSS protection, CSP enforcement, and third-party script management**, with high potential for authenticated account compromise and data leakage. While medium and low risks are less likely to be exploited directly, they cumulatively increase the attack surface and can aid attackers in reconnaissance and exploitation.

Tools Used

1. **Nmap** : For port scanning and service/version enumeration.
2. **Burp Suite** : For intercepting HTTP requests, analyzing cookies, testing CSRF, XSS, and CSP weaknesses.
3. **OWASP ZAP** : Automated vulnerability scanning and security header analysis.
4. **Browser Developer Tools** : Inspecting meta tags, cookies, headers, and inline scripts.
5. **CSP Evaluators / Security Headers Checkers** : To analyze Content Security Policy, HSTS, X-Frame-Options, Referrer-Policy, Permissions-Policy.
6. **Wappalyzer / BuiltWith** : Technology stack and third-party service detection.
7. **Manual Testing** : Verification of clickjacking, iframe behavior, CSRF token exposure, and telemetry leaks.

🔴 Critical Risks

1. **CSRF Token Exposure / XSRF Cookie Readable by JS**
    - **Evidence:** `<meta name="csrf-token"content="...">` and `Set-Cookie: XSRF-TOKEN=...;SameSite=None` (no HttpOnly)

- **Impact:** Any XSS or third-party script can read tokens → CSRF protection bypassed → authenticated actions possible.
- **Recommendation:** Store CSRF tokens in HttpOnly cookies, validate Origin/Referer server-side, reduce third-party JS, rotate tokens frequently.

2. **Inline Script Nonce Reuse / Predictable Nonce**
   - **Evidence:** `<script nonce="...">` observed
   - **Impact:** Attackers may bypass CSP → XSS possible.
   - **Recommendation:** Generate cryptographically random nonce per response; prefer external JS + strict CSP.
3. **CSP Permissive & Report-Only**
   - **Evidence:** `Content-Security-Policy-Report-Only` with unsafe-inline, unsafe-eval, *, many third-party domains
   - **Impact:** XSS and malicious third-party scripts can execute; CSP ineffective.
   - **Recommendation:** Harden CSP, remove unsafe directives, restrict domains, enforce after testing.

High Risks

1. **Clickjacking Allowed**
   - **Evidence:** Page can be iframed
   - **Impact:** User could be tricked into unintended actions → account takeover.
   - **Recommendation:** Apply `X-Frame-Options: DENY` or CSP `frame-ancestors 'none'`.
2. **Credentialed Requests in Iframes**
   - **Evidence:** `/api/monitor` sends `fetch(..., credentials: "include")` inside iframe
   - **Impact:** Malicious site could trigger authenticated actions.
   - **Recommendation:** Validate CSRF, Origin, Referer headers; restrict credentials in iframes.
3. **Permissive Third-Party Domains / Wildcards**
   - **Evidence:** CSP allows many external services (`*.domain`)

- **Impact:** Supply-chain compromise → arbitrary code execution.
  Recommendation: Restrict to trusted domains, use vetted proxy/CDN, load
- non-essential scripts after consent.

4. **Permissive form-action**
   - **Evidence:** `form-action 'self'*`
   - **Impact:** Forms may submit to attacker-controlled endpoints → data exfiltration.
   - **Recommendation:** Restrict to trusted endpoints only.
5. **Cookies with SameSite=None**
   - **Evidence:** `XSRF-TOKEN` and `laravel_session` use `SameSite=None`
   - **Impact:** Cross-site requests include cookies → higher CSRF risk.
   - **Recommendation:** Use SameSite=Lax/Strict; strengthen server-side CSRF checks.

Medium Risks

1. **Missing / Weak Security Headers**
   - **Evidence:** No full CSP, HSTS lacks includeSubDomains/preload, `missing Referrer-Policy, Permissions-Policy`
   - **Impact:** Increases XSS, downgrade attacks, referrer leakage, feature misuse.
   - **Recommendation:** Set HSTS, Referrer-Policy, Permissions-Policy, X-Content-Type-Options, CSP with nonces.
2. **Telemetry & Analytics Risks**
   - **Evidence:** GTM/Telemetry scripts send credentials: include and parent/referrer URL
     Impact: Sensitive cookies or URLs may leak; supply-chain risk.
   - **Recommendation:** Send only sanitized origin, avoid credentials in telemetry, audit third-party scripts, load after consent.
   -
3. **Open Ports / Service Disclosure**
   - **Evidence:** Ports 80/443 open; nginx version and OS leaked

- **Impact:** Facilitates fingerprinting, automated attacks, potential outdated service exploits.
  Recommendation: Enforce HTTPS, hide versions, patch services, firewall
- hardening, routine scans.

4. **No WAF**
   - **Evidence:** No detectable WAF
   -
   - **Impact:** Direct exposure to attacks (SQLi, XSS, brute force)

     Recommendation: Deploy WAF, implement input validation, rate-limiting, monitoring.

Low Risks

1. **Server Information Disclosure**
   - **Evidence:** `Server: nginx` header
   - **Impact:** Fingerprinting; minor risk if patched
   - **Recommendation:** Hide/generalize server banner, patch regularly.
2. **Telemetry / Referrer Leakage**
   - **Evidence:** `sendToMonitoring(getParentUrl())` sends parent URL
   - **Impact:** May expose sensitive query strings → privacy/GDPR risk
   - **Recommendation:** Send only origin, strip query params/PII.
3. **Cookie Security Attributes Missing**
   - **Evidence:** Cookies observed without Secure/HttpOnly/SameSite flags
   - **Impact:** Risk of theft via XSS/CSRF
   - **Recommendation:** Set `HttpOnly; Secure; SameSite=Lax` on all sensitive cookies.

Risk Level	Finding / Evidence	Impact	Recommendation
🔴 Critical	CSRF token in <meta> + XSRF-TOKEN cookie readable by JS	XSS or third-party scripts can bypass CSRF → authenticated actions	Store CSRF in HttpOnly cookies, validate Origin/Referer, reduce 3rd-party JS, rotate tokens frequently
🔴 Critical	Inline script nonce predictable / reused	CSP bypass → XSS possible	Generate random nonce per response; prefer external JS + strict CSP
🔴 Critical	CSP Report-Only, permissive (`unsafe-inline`, `unsafe-eval`, `*`)	XSS & malicious scripts execute; CSP ineffective	Harden CSP, remove unsafe directives, restrict domains, enforce after testing
🔴 High	Clickjacking allowed (page can be iframed)	Users may click hidden elements → account takeover	Apply `X-Frame-Options: DENY` or CSP `frame-ancestors 'none'`
🔴 High	Credentialed requests in `iframe (fetch(..., credentials: "include"))`	Malicious site may trigger authenticated actions	Validate CSRF/Origin/Referer; restrict credentials in iframes
🔴 High	Permissive 3rd-party domains / wildcards in CSP	Compromise of vendor → arbitrary code execution	Restrict to minimal trusted domains, use vetted proxy/CDN, load non-essential scripts after consent
🔴 High	Permissive form-action (`'self' *`)	Forms may submit to attacker endpoints → data exfiltration	Restrict form-action to trusted endpoints only
🔴 High	Cookies with SameSite=None (`XSRF-TOKEN`, `laravel_session`)	Cross-site requests include cookies → higher CSRF risk	Use SameSite=Lax/Strict; strengthen server-side CSRF validation
🔴 Medium	Missing/weak security headers (CSP, HSTS, Referrer-Policy, Permissions-Policy, X-Content-Type-Options)	XSS, downgrade attacks, referrer leakage, feature misuse	Set HSTS, Referrer-Policy, Permissions-Policy, X-Content-Type-Options, CSP with nonce
🔴 Medium	Telemetry / analytics sending `credentials: include` + parent/referrer	Sensitive cookies/URLs may leak; supply-chain risk	Send only sanitized origin, avoid credentials in telemetry, audit 3rd-party scripts, load after consent

🔴 Medium	Open ports 80/443, nginx version & OS disclosed	Fingerprinting, automated attacks, potential outdated exploits	Enforce HTTPS, hide versions, patch services, firewall hardening, routine scans
🔴 Medium	No WAF detected	Direct exposure to attacks (SQLi, XSS, brute force)	Deploy WAF, input validation, rate-limiting, monitoring
🔴 Low	Server header disclosure (`Server: nginx`)	Fingerprinting (minor risk if patched)	Hide/generalize server banner, patch regularly
🔴 Low	Telemetry leaks parent/referrer URL	Exposes sensitive query strings → privacy/GDPR risk	Send only origin, strip query params/PII
🔴 Low	Cookies missing Secure/HttpOnly/SameSite	Theft via XSS/CSRF possible	Set `HttpOnly; Secure; SameSite=Lax` on all sensitive cookies

---

Final Recommendations

1. **CSRF & Session Protection**
   - Store CSRF tokens in HttpOnly cookies; validate Origin/Referer server-side.
   - Enforce SameSite=Lax/Strict for session and CSRF cookies.
   - Reduce exposure to third-party JS and rotate tokens frequently.
2. **Content Security Policy & Script Management**
   - Harden CSP: remove unsafe-inline/unsafe-eval, restrict domains, use per-response nonces or hashes.
   - Move inline scripts to external, trusted sources.
   - Monitor/report-only CSP during testing, then enforce once validated.
3. **Clickjacking & Iframe Security**
   - Apply `X-Frame-Options: DENY` or CSP `frame-ancestors 'none'`.
   - Validate CSRF, Origin, and Referer for iframe-initiated requests.
4. **Third-Party & Telemetry Controls**
   - Restrict allowed third-party domains; use vetted proxy/CDN.

- Load non-essential scripts after user consent.
- Sanitize telemetry/referrer data; avoid sending credentials in telemetry.

5. **Infrastructure & Headers**
   - Deploy a WAF (Cloud or on-prem) to mitigate automated attacks.
   - Enforce HTTPS, enable HSTS with includeSubDomains and `preload`.
   - Set missing headers: Referrer-Policy, Permissions-Policy, X-Content-Type-Options.
   - Hide server information and remove version disclosures.
6. **Ongoing Security Measures**
   - Conduct routine vulnerability scans and penetration tests.
   - Patch web server, OS, and dependencies regularly.
   - Implement monitoring/alerting for abnormal activity or suspicious requests.

Conclusion:
Immediate remediation of **critical and high-risk vulnerabilities** is essential to prevent account compromise, CSRF/XSS exploitation, and supply-chain attacks. Medium and low-risk issues should be addressed systematically to strengthen overall security posture and ensure regulatory compliance.
